

Java 高速语法复习手册

从概念 → 代码 → 坑，覆盖基本类型、控制语句、OOP 重点（动态绑定、抽象类、接口、内部类、接口实例化）、常用数据结构。

一、基本数据类型

1. 8 种基本类型

类型	大小	默认值	包装类
byte	1B	0	Byte
short	2B	0	Short
int	4B	0	Integer
long	8B	0L	Long
float	4B	0.0f	Float
double	8B	0.0d	Double
char	2B	'\u0000'	Character
boolean	JVM 相关	false	Boolean

2. 代码与坑

```
// 自动装箱/拆箱
Integer a = 127;
Integer b = 127;
System.out.println(a == b); // true, 因为 -128~127 缓存

Integer c = 128;
Integer d = 128;
System.out.println(c == d); // false, 超出缓存范围, 不同对象

// 类型转换
int i = 10;
long l = i;           // 隐式转换, 小转大
int j = (int) l;       // 强制转换, 可能丢失精度

// float 赋值必须加 f
float f = 3.14f;
double d2 = 3.14;     // 默认 double

// char 与 int 互转
char ch = 'A';
int code = ch;         // 65
```

⚠ 坑: 包装类比较尽量用 `equals()`, `==` 比较地址, 容易翻车。 `boolean` 不能与数字互转, `String` 不是基本类型。

二、控制语句

1. 条件分支

```
// if-else
if (x > 0) { ... } else if (x == 0) { ... } else { ... }

// switch 支持: int/char/String/枚举 ( byte short 自动提升 )
switch (day) {
    case "MON":
        // 注意穿透, 除非故意否则必须写 break
        break;
    case "TUE":
        break;
    default:
        break;
}
```

⚠ 坑: case 穿透不写 break 会继续执行下一个 case; switch 表达式的值不能是 long / float / double / boolean 。

2. 循环

```
// 普通 for
for (int i = 0; i < 10; i++) { ... }

// 增强 for-each (适用于数组和 Iterable)
for (String s : list) { ... }

// while / do-while
while (condition) { ... }
do { ... } while (condition);
```

3. 跳转

```
break;           // 跳出当前循环/switch
continue;        // 跳过当前迭代剩余部分
return;          // 退出方法

// 带标签的跳转
outer:
for (...) {
    for (...) {
        break outer; // 直接跳出外层循环
    }
}
```

三、面向对象编程（重点）

1. 核心概念速览

- **封装**： private 字段 + public getter/setter
- **继承**： extends ，单继承，构造器链(super())
- **多态**：父类引用指向子类对象，方法调用动态绑定

2. 动态绑定（动态数据绑定）

定义：方法调用在 **运行时** 根据对象的实际类型决定执行哪个方法，而不是引用类型。字段没有多态。

```
class Animal {
    String name = "Animal";
    void speak() { System.out.println("Animal speaks"); }
}

class Dog extends Animal {
    String name = "Dog";
    @Override void speak() { System.out.println("Dog barks"); }
}

public class Test {
    public static void main(String[] args) {
        Animal a = new Dog();
        a.speak();           // 输出 Dog barks    (动态绑定)
        System.out.println(a.name); // 输出 Animal (字段没有重写，看引用类型)
    }
}
```

⚠ 坑：

- 字段不参与多态，访问时看引用类型，容易误解。
- 静态方法不能重写，只能隐藏，没有动态绑定。
- 重写时返回值类型可以是父类方法返回类型的子类（协变返回类型）。
- 重写方法不能有更严格的访问权限（`public` → `private` 不行）。

3. 抽象类

```
abstract class Shape {  
    protected String color;  
    // 抽象类可以有构造方法，供子类调用  
    public Shape(String color) { this.color = color; }  
    // 抽象方法，子类必须实现  
    public abstract double area();  
    // 普通方法  
    public void printColor() { System.out.println(color); }  
}  
  
class Circle extends Shape {  
    double radius;  
    public Circle(String color, double r) {  
        super(color);  
        this.radius = r;  
    }  
    @Override  
    public double area() { return Math.PI * radius * radius; }  
}
```

⚠ 坑：

- 抽象类 **不能实例化**（`new Shape()` 错误）。
- 有抽象方法的类必须是抽象类，但抽象类可以没有抽象方法。
- 子类若不实现所有抽象方法，子类也必须声明为 `abstract`。

4. 接口

```
interface Flyable {
    // 常量（默认 public static final）
    int MAX_SPEED = 300; // 等价 public static final int MAX_SPEED = 300;

    // 抽象方法（默认 public abstract）
    void fly();

    // Java 8 默认方法，可提供实现
    default void land() {
        System.out.println("Landing...");
    }

    // Java 8 静态方法
    static void info() {
        System.out.println("Flyable interface");
    }
}

// 类可实现多个接口
class Bird implements Flyable {
    @Override
    public void fly() { System.out.println("Bird flying"); }
}
```

⚠ 坑：

- 接口不能持有实例字段（只能有常量）。
- 接口不能实例化，但常用**匿名内部类**或 **Lambda** 实现（见下节）。
- 一个类实现多接口，如果有冲突的默认方法，必须重写解决。
- 接口中的方法默认 `public`，实现时不能降低可见性。

5. 内部类与接口实例化

① 成员内部类（非静态）

```
class Outer {  
    private int x = 10;  
    class Inner {  
        void show() { System.out.println(x); } // 可直接访问外部类成员  
    }  
}  
// 实例化必须通过外部类对象  
Outer out = new Outer();  
Outer.Inner in = out.new Inner();  
// 或者一步: Outer.Inner in = new Outer().new Inner();
```

⚠ 成员内部类会持有外部类的引用，可能导致内存泄漏。

② 静态内部类

```
class Outer {  
    static class StaticInner { ... }  
}  
Outer.StaticInner si = new Outer.StaticInner(); // 不需要外部对象
```

③ 局部内部类（方法内）

```
public void method() {  
    final int localVar = 10; // 实际 java 8 起可省略 final，但必须 effectively final  
    class LocalInner {  
        void print() { System.out.println(localVar); }  
    }  
    LocalInner li = new LocalInner(); // 只能在方法内使用  
}
```

⚠ 局部内部类只能访问 final 或 effectively final 的局部变量。

④ 匿名内部类 → 接口实例化

接口本身不能 new，但通过匿名内部类可以创建一个实现该接口的对象：


```
Flyable f = new Flyable() {  
    @Override  
    public void fly() {  
        System.out.println("Anonymous fly");  
    }  
};  
f.fly();
```

函数式接口（只有一个抽象方法）可以用 Lambda 简化：

```
// 函数式接口  
@FunctionalInterface  
interface Calculator {  
    int compute(int a, int b);  
}  
  
// Lambda 表达式  
Calculator add = (a, b) -> a + b;  
System.out.println(add.compute(3, 5));
```

⚠ 坑：

- 匿名内部类与 Lambda 的 `this` 含义不同：匿名内部类中 `this` 指向匿名内部类自身对象；Lambda 中 `this` 指向外层类实例。
- Lambda 要求接口必须是函数式接口，否则编译错误。
- 匿名内部类会造成额外 `.class` 文件，大量使用可能影响性能。

四、算法常用数据结构（集合框架）

1. List（列表） — 有序、可重复

ArrayList（动态数组）

```
List<Integer> list = new ArrayList<>();
list.add(1); list.add(2);
list.get(0);           // 1
list.set(0, 10);
list.remove(1);         // 按索引删除，返回被删元素
list.remove(Integer.valueOf(10)); // 按对象删除（注意装箱）
list.size();
for (int x : list) { ... }
```

⚠ 坑：

- `remove(int index)` 与 `remove(Object o)` 重载：想删除整数对象时要写 `remove(Integer.valueOf(5))`，否则变成按索引删除。
- 初始容量 10，扩容为 1.5 倍，频繁扩容影响性能，可预估大小时用 `new ArrayList<>(capacity)`。

LinkedList（双向链表，也实现了 Deque）

```
LinkedList<String> linkedList = new LinkedList<>();
linkedList.addFirst("a");
linkedList.addLast("b");
linkedList.removeFirst();
linkedList.getFirst();
```

随机访问 `get(index)` 慢 $O(n)$ ，适合频繁头尾操作。

2. Set（集合） — 无重复

HashSet（基于 HashMap，无序）

```
Set<String> set = new HashSet<>();
set.add("apple");
set.contains("apple");
set.remove("apple");
```

⚠️ 必须同时重写 `equals()` 和 `hashCode()`，否则自定义对象去重失效。

TreeSet（红黑树，自动排序）

```
Set<Integer> treeSet = new TreeSet<>();
treeSet.add(3); treeSet.add(1); // 自动排序 [1,3]
// 自定义对象必须实现 Comparable 或提供 Comparator
Set<Person> people = new TreeSet<>(Comparator.comparing(Person::getAge));
```

⚠️ TreeSet 不能放入 `null`（需要比较）。HashSet 允许一个 `null`。

LinkedHashSet（保持插入顺序）

```
Set<Integer> linked = new LinkedHashSet<>();
```

3. Map（映射） — 键值对

HashMap（数组+链表/红黑树）

```
Map<String, Integer> map = new HashMap<>();
map.put("key", 1);
map.get("key");           // 不存在返回 null
map.containsKey("key");
map.remove("key");
// 遍历
for (Map.Entry<String, Integer> entry : map.entrySet()) {
    String k = entry.getKey();
    Integer v = entry.getValue();
}
// 只遍历键或值
map.keySet();    // Set<K>
map.values();    // Collection<V>
```

⚠ 重要坑：

- 自定义对象作为键，必须重写 `hashCode()` 和 `equals()`，且 key 最好不可变。
- HashMap 允许 key/value 为 `null`，HashTable 不允许。
- 并发修改用 `ConcurrentHashMap`，普通 `HashMap` 不是线程安全。

TreeMap（红黑树，按键排序）

```
Map<Integer, String> treeMap = new TreeMap<>();
treeMap.put(3, "c"); treeMap.put(1, "a"); // 键自动排序
// 自定义排序：
Map<Person, String> personMap = new TreeMap<>(Comparator.comparing(Person::getAge));
```

4. 栈和队列（算法高频）

Stack（遗留类，推荐用 Deque）

```
Deque<Integer> stack = new ArrayDeque<>();
stack.push(1);        // 压栈
stack.peek();         // 查看栈顶
stack.pop();          // 弹栈
```

队列 (Queue)

```
Queue<Integer> queue = new LinkedList<>(); // LinkedList 实现了 Queue
queue.offer(1);           // 入队
queue.poll();             // 出队, 为空返回 null
queue.peek();             // 查看队头
```

双端队列 (Deque, 可充当栈/队列)

```
Deque<Integer> deque = new ArrayDeque<>();
deque.addFirst(1); deque.addLast(2);
deque.removeFirst(); deque.removeLast();
```

优先队列 (堆, 默认小顶堆)

```
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
// 大顶堆
PriorityQueue<Integer> maxHeap = new PriorityQueue<>((a,b) -> b - a);
// 或者 Comparator.reverseOrder()
maxHeap.offer(5); maxHeap.offer(2);
maxHeap.poll(); // 弹出最大 5
```

⚠️ PriorityQueue 自定义对象必须提供 Comparator, 否则抛 ClassCastException。

5. 遍历中的注意事项

```
// 用迭代器安全删除
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    String s = it.next();
    if (shouldRemove(s)) {
        it.remove(); // 不要用 list.remove(s) 会 ConcurrentModificationException
    }
}

// Java 8 removeIf
list.removeIf(s -> s.startsWith("A"));
```

五、总结速查表

需求	推荐类	说明
快速随机访问列表	ArrayList	O(1) get
频繁头尾增删	LinkedList / ArrayDeque	双端队列
去重，无序	HashSet	必须重写 hashCode>equals
去重，排序	TreeSet	元素可比较
键值映射，高性能	HashMap	key 重写 hashCode>equals
按键排序	TreeMap	红黑树
栈	ArrayDeque (push/pop)	替代 Stack
堆/优先队列	PriorityQueue	默认小顶堆，可传比较器
函数式接口实现	Lambda 或匿名内部类	接口实例化的唯一途径

用这些代码片段和坑点快速唤醒记忆，剑指高频场景。